

수식	의미	변수 설명
$CPU\ Time = IC \times CPI \times T_c$	CPU 실행 시간	IC=명령어 수, CPI=명령어당 사이클, T_c =클럭 주기
$CPU\ Time = \frac{Clock\ Cycles}{Clock\ Rate}$	위와 동치	$Clock\ Cycles = IC \times CPI$, $Clock\ Rate = 1/T_c$
$Performance = 1 / Execution\ Time$	성능 정의	
$Speedup = \frac{Perf_X/Perf_Y = Time_Y/Time_X}$	X가 Y보다 몇 배 빠른가	
$CPI = \frac{\sum(CPI_i \times IC_i)}{IC_total}$	가중 평균 CPI	CPI_i =클래스별 CPI, IC_i =클래스별 명령어 수
$Power = C \times V^2 \times f$	CMOS 동적 전력	C =capacitive load, V =전압, f =주파수
$T_{improved} = T_{affected}/n + T_{unaffected}$	Amdahl's Law	n =개선 배율
$Cost/die = \frac{Cost_wafer}{(Dies/wafer \times Yield)}$	다이 비용	다이 면적↑ → 비용 비선형↑
$Yield \approx 1/(1 + Defects \times Die_area/2)^2$	수율 근사	
$SPEC\ ratio = \frac{Ref_time}{Target_time}$	SPEC 점수	높을수록 빠름
$Overall\ SPEC = (\prod ratio_i)^{(1/n)}$	SPEC 기하 평균	n =벤치마크 수
Moore's Law: 현재 $\times 2^{(n/2)}$	n 년 후 트랜지스터 수	
$lll\ by\ i = \times 2^i$	좌 시프트 = 곱셈	
$rll\ by\ i = \div 2^i$	우 논리 시프트 = 나눗셈	
Byte offset = index $\times 8$	doubleword 배열 주소	$A[i] \rightarrow offset = i \times 8$
$TCO = CAPEX + OPEX$	총 소유 비용	CAPEX=초기 투자, OPEX=운영 비용
$perf/TCO$	TCO 단위당 성능	$perf$ =성능(throughput 등), TCO=총 소유 비용
$FP_{값} = (-1)^S \times (1+Frac) \times 2^{(Exp-Bias)}$	IEEE 754 부동소수점 값	S =부호, $Frac$ =소수부, Exp =지수, $Bias=127(single)/1023(double)$
Single range: $\pm 1.2 \times 10^{-38} \sim \pm 3.4 \times 10^{38}$	단정밀도 범위	정밀도 ~6자리 십진수 (2^{-23})
Double range: $\pm 2.2 \times 10^{-308} \sim \pm 1.8 \times 10^{308}$	배정밀도 범위	정밀도 ~16자리 십진수 (2^{-52})

$\text{Speedup_pipeline} \approx \text{Stages (이상적)}$	파이프라인 이상적 속도 향상	$\text{Stages} = \text{파이프라인 단계 수}$
$\text{IPC} = 1/\text{CPI}$	사이클당 명령어 수	Multiple issue에서 $\text{CPI} < 1$ 일 때 사용
$\text{AMAT} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$	평균 메모리 접근 시간	
$\text{Mem stall} = \text{IC} \times (\text{Misses/Inst}) \times \text{Miss penalty}$	메모리 stall 사이클	I-cache + D-cache 별도
$\text{CPI_actual} = \text{CPI_base} + \text{Mem_stall/Inst}$	실제 CPI	캐시 miss 반영
$\text{MTBF} = \text{MTTF} + \text{MTTR}$	평균 고장 간격	
$\text{Availability} = \text{MTTF} / (\text{MTTF} + \text{MTTR})$	가용성	1에 가까울수록 좋음
$\text{Set index} = \text{Block_addr mod Num_sets}$	Set-assoc 인덱스	Direct: $\text{Num_sets} = \text{Num_blocks}$
$\text{Block_addr} = \text{floor}(\text{Byte_addr} / \text{Block_size})$	블록 주소	$\text{Offset} = \text{Byte_addr mod Block_size}$
$\text{Speedup_parallel} = 1 / ((1-F) + F/N)$	Amdahl 병렬 버전	$F = \text{병렬화 비율}$, $N = \text{프로세서 수}$
$\text{Attainable GFLOP} = \text{Min}(\text{BW} \times \text{AI}, \text{Peak})$	Roofline 모델	$\text{BW} = \text{메모리대역폭}$, $\text{AI} = \text{FLOPs/Byte}$
$\text{Buffer Size} = \text{Bandwidth} \times \text{Latency}$	Little's Law	BW를 100% 활용하려면 이만큼의 in-flight 요청 필요

비교 항목	A	B
ISA vs ABI	HW/SW 인터페이스(명령어 집합)	ISA + 시스템SW 인터페이스
ISA vs Implementation	명세("무엇을")	마이크로아키텍처("어떻게")
RISC vs CISC	단순·규칙·고정길이	복잡·가변길이
Response Time vs Throughput	작업 완료 시간	단위시간당 총 작업량
Elapsed vs CPU Time	전체응답(I/O, OS포함)	순수 연산만
Volatile vs Non-volatile	DRAM(전원off시 소실)	HDD, SSD, Flash(보존)
SRAM vs DRAM	빠르고 비쌈(캐시)	느리고 저렴(메인메모리)
Saved vs		

Temporary regs	x8-9,x18-27(callee보존)	x5-7,x28-31(비보존)
Leaf vs Non-leaf	다른 함수 호출 안 함	다른 함수 호출함
lb vs lbu	Sign extend	Zero extend
blt/bge vs bltu/bgeu	Signed 비교	Unsigned 비교
Static vs Dynamic linking	컴파일시 라이브러리 포함	실행시 필요할때 로드
Array vs Pointer	매번 $\text{index} \times \text{size} + \text{base}$	주소 직접 증가
PC-relative vs Absolute	$\text{PC} + \text{offset}(\text{분기})$	전체주소(lui+jalr)
CAPEX vs OPEX	초기 투자비(서버·장비 구매)	운영비(전력·냉각·인건비 유지보수)
Single vs Double precision	32-bit($\text{S1} + \text{Exp8} + \text{Frac23}$), Bias=127	64-bit($\text{S1} + \text{Exp11} + \text{Frac52}$), Bias=1023
FP 덧셈 vs FP 곱셈	지수 정렬→가수 덧셈→정규화→반올림	지수 덧셈→가수 곱셈→정규화→반올림→부호결정
Restoring vs Non-restoring div	음수 나머지 시 divisor 다시 더함	다음 단계에서 보정
Guard/Round/Sticky bits	추가 정밀도 비트	
Integer regs vs FP regs	x0~x31 (32×64-bit)	f0~f31 (32×64-bit)
Single-cycle vs Pipelined	한 사이클에 명령어 완료, $T_c = \text{최긴 경로}$	5 stage 중첩, $T_c = \text{최긴 stage}$
Forwarding vs Stalling	결과를 즉시 전달(bypass)	1+cycle 대기(bubble 입)
Static vs Dynamic branch pred	컴파일러가 방향 예측 (backward taken 등)	HW가 BHT로 런타임 0 (1-bit, 2-bit)
Static vs Dynamic multiple issue	컴파일러가 issue slot 패킹 (VLIW)	CPU가 런타임에 발행 (Superscalar)
In-order vs Out-of-order	프로그램 순서대로 실행 (A53)	의존성 없으면 먼저 실행 (i7)
Exception vs Interrupt	CPU 내부 발생 (opcode, overflow, syscall)	외부 I/O 컨트롤러
ARM A53 vs Intel i7	PMD, 100mW, 8-stage in-order	Server, 130W, 14-stage OoO+speculation
Write-Through vs Write-Back	캐시+메모리 동시 갱신	캐시만 갱신, dirty시 교체 쓰기
Direct Mapped vs		

Set-Assoc vs Fully	1곳/n곳/어디든	비교기 1/n/전체
SRAM vs DRAM vs Flash vs Disk	0.5ns,\$5K/GB / 50ns,\$6/GB / 50μs,\$0.75/GB / 10ms,\$0.07/GB	
Temporal vs Spatial locality	최근 접근→곧 재접근	인접 데이터→곧 접근
L1 vs L2 cache	작고 빠름(hit time 최소화)	크고 느림(miss rate 최소화)
Compulsory vs Capacity vs Conflict	첫 접근 / 캐시 용량 부족 / set 내 충돌	
Page Table vs TLB	메모리의 전체 매핑 테이블	CPU 내 PTE 캐시 (16~512)
Snooping vs Directory coherence	모든 캐시가 버스 감시	공유 상태를 디렉토리 가
Strong vs Weak scaling	문제 크기 고정, 프로세서 ↑	문제도 프로세서에 비례
Fine-grain vs Coarse-grain MT	매 사이클 스레드 전환	긴 stall(L2 miss)에서 단
SMT vs Coarse-grain MT	같은 사이클에 여러 스레드 혼합	stall 시만 전환
Shared Memory vs Message Passing	공유 물리 주소 공간, lock 동기화	독립 주소 공간, send/receive
UMA vs NUMA	모든 프로세서 메모리 접근 동일	로컬이 더 빠름
CPU vs GPU	큰 캐시, coherent, 다양한 워크로드	작은 캐시, 대규모 스레드, 높은 BW
Vector vs SIMD(SSE/AVX)	가변 폭, strided 접근, 컴파일러 친화	고정 폭(128/256b), stride 미지원
Memory bound vs Compute bound	AI < 교차점 (메모리 BW 제한)	AI > 교차점 (FP 성능 가
FEC vs ARQ	수신 측이 자체 정정(재전송 불필요)	에러 검출 시 재전송 요
Detected Error vs SDC	ECC 등이 에러를 검출→정정/재시도	에러가 검출 안 됨→잘못 결과 전파

용어	정의
ABI (Application Binary Interface)	ISA + 시스템SW 인터페이스. 같은 ISA라도 OS 다르면 ABI 다를 수 있음
Abstraction	하위 세부사항을 숨기고 단순 인터페이스만 노출하는 설계 원칙
Amdahl's Law	부분 개선이 전체 성능에 비례하지 않음. $T = T_{\text{affected}}/n + T_{\text{unaffected}}$

AMAT	Average Memory Access Time = Hit time + Miss rate × Miss penalty
Basic Block	중간에 분기 없고 분기대상도 처음뿐인 명령어 시퀀스. 최적화 기본 단위
Cache Memory	CPU 내/근처 소형 고속 SRAM. 메인 메모리 접근 지연 감소
Cache Coherence	멀티프로세서 공유 데이터 일관성. Snooping(버스 감시)/Directory(상태 기록)
Coherence vs Consistency	Coherence: 같은 주소의 읽기가 최신 쓰기 값을 반환. Consistency: 서로 다른 주소의 쓰기가 다른 프로세서에 보이는 순서 규칙
CAPEX (Capital Expenditure)	자본적 지출. 서버·장비·인프라 등 초기 구매/구축 비용. 일회성 투자
CISC (Complex Instruction Set Computer)	복잡·다양·가변길이 명령어. x86 대표
CPI (Cycles Per Instruction)	명령어당 평균 클럭 사이클. CPU HW와 명령어 mix에 의해 결정
Datapath	프로세서 내부 데이터 연산 유닛 (ALU, 레지스터)
Denormal Number	Exponent=0일 때 hidden bit=0. 정규화 수보다 더 작은 값 표현. Gradual underflow 허용
Control	명령어 해석(decode) → Datapath에 제어 신호
Chaining	벡터 프로세서에서 한 벡터 연산의 결과를 다음 연산의 입력으로 직접 전달. 스칼라 forwarding의 벡터 버전. 원소 단위로 파이프라인 연결하여 전체 완료를 기다리지 않음
Design Principle 1	Simplicity favors regularity
Design Principle 2	Smaller is faster
Design Principle 3	Good design demands good compromises
DLP (Data-Level Parallelism)	같은 연산을 여러 데이터에 동시 적용. SIMD, Vector, GPU가 활용. 배열/행렬/이미지 처리에 풍부. Flynn SIMD
DMA (Direct Memory Access)	CPU 거치지 않고 메모리↔I/O 간 직접 전송. CPU는 전송 중 다른 일 수행 가능. GPU↔Host 대량 전송에 사용
Dynamic Linking	호출 시점에만 라이브러리 로드. Lazy Linkage
Exception	CPU 내부에서 발생하는 예외 (undefined opcode, overflow, syscall). SEPC/SCAUSE에 기록 후 handler로 점프
Forwarding (Bypassing)	파이프라인에서 결과가 나오면 레지스터 write 전에 바로 다음 명령어 입력으로 전달. Data hazard 해결
GPU (Graphics Processing Unit)	대규모 병렬 데이터 처리 프로세서. 수천 스레드, 높은 메모리 BW. CUDA/OpenCL로 GPGPU 활용

Hazard	파이프라인에서 다음 명령어를 즉시 시작하지 못하게 하는 상황. Structural/Data/Control 3종류
IC (Instruction Count)	프로그램 실행 시 총 명령어 수
IEEE 754	부동소수점 표현 및 연산의 국제 표준. Single(32-bit), Double(64-bit). Bias, hidden bit, 특수값(± 0 , $\pm \text{Inf}$, NaN, Denormal) 정의
ILP (Instruction-Level Parallelism)	단일 프로그램 내 여러 명령어를 동시 실행. Pipeline, OoO, VLIW로 활용. HW가 자동 추출. 의존성이 한계 결정
Infinity ($\pm \infty$)	Exp=111...1, Frac=000...0. 오버플로우 시 사용. 후속 계산에 전파 가능
ISA (Instruction Set Architecture)	HW/SW 인터페이스. 프로세서가 이해하는 명령어 집합 정의
ILP (Instruction-Level Parallelism)	파이프라인/다중 발행으로 여러 명령어를 동시 실행하는 병렬성
IPC (Instructions Per Cycle)	사이클당 명령어 수. $\text{CPI} < 1$ 일 때 (multiple issue) 사용. $\text{IPC} = 1/\text{CPI}$
Little Endian	최하위 바이트가 가장 낮은 주소. RISC-V 사용
Little's Law	Buffer Size = Bandwidth $\times$ Latency. 메모리 BW를 100% 활용하려면 latency 동안 충분한 in-flight 요청이 필요. GPU가 수천 스레드를 유지하는 이론적 근거
Locality	Temporal(최근 $\rightarrow$ 재접근) + Spatial(인접 $\rightarrow$ 접근). 메모리 계층의 이론적 근거
MIMD (Multiple Instruction Multiple Data)	Multiple Instruction Multiple Data. 여러 프로세서가 독립 명령어로 독립 데이터 처리. 멀티코어
NUMA (Non-Uniform Memory Access)	Non-Uniform Memory Access. 로컬 메모리가 더 빠른 공유 메모리 구조. Memory affinity 중요
lr.d / sc.d	Load Reserved / Store Conditional. 동기화용 atomic 쌍
lui	20-bit 상수를 rd[31:12]에 로드, sign extend, [11:0]=0
Moore's Law	IC 트랜지스터 수 약 2년마다 2배. 경험적 관찰(물리법칙 아님)
NaN (Not a Number)	Exp=111...1, Frac $\neq 0$. 0/0 등 정의되지 않은 결과. 후속 계산에 전파
OPEX (Operating Expenditure)	운영 지출. 전력·냉각·인건비·유지보수·네트워크 등 지속적 비용. 데이터센터에서 전력이 OPEX의 핵심
Overflow	연산 결과가 표현 가능 범위 초과. 정수: 부호 반전으로 감지. FP: Infinity로 처리
Power Wall	전압/발열 한계로 클럭 향상 불가 $\rightarrow$ 멀티코어 전환
Pipeline	명령어 실행을 여러 stage로 나누어 중첩 실행. Throughput $\uparrow$, Latency

	동일
Pipeline Register	각 stage 사이의 레지스터. 이전 cycle 정보 보존 (IF/ID, ID/EX, EX/MEM, MEM/WB)
R/I/S/SB/U/UJ-type	RISC-V 6가지 32-bit 고정길이 명령어 포맷
RISC-V	UC Berkeley 개발 오픈 RISC ISA. 이 과목 기준 ISA
Roofline Model	Attainable GFLOP = $\text{Min}(\text{BW} \times \text{AI}, \text{Peak FP})$. AI로 memory/compute bound 판별
SMT (Simultaneous Multithreading)	다중 발행 프로세서에서 여러 스레드의 명령어를 동시 스케줄링. Intel HT
SPMD (Single Program Multiple Data)	Single Program Multiple Data. MIMD에서 같은 프로그램을 조건문으로 분기. 가장 흔한 병렬 모델
Saturating Operation	오버플로우 시 wrap-around 대신 최대/최소값으로 고정. 오디오/비디오 처리에 사용
SIMD (Single Instruction Multiple Data)	Single Instruction Multiple Data. 하나의 명령어로 여러 데이터를 동시 처리. SSE/AVX가 대표적
Sign Extension	넓은 비트 확장 시 부호비트 복제. lb(sign), lbu(zero)
SPEC (Standard Performance Evaluation Corporation)	Standard Performance Evaluation Corp. 기하평균 벤치마크
Stored Program	명령어도 이진수로 메모리 저장. 프로그램이 프로그램 조작 가능
TLP (Thread-Level Parallelism)	여러 스레드를 동시 실행. 멀티코어, SMT가 활용. 프로그래머가 명시적 병렬화 필요(OpenMP, pthread). Flynn MIMD
TLB (Translation Lookaside Buffer)	CPU 내 PTE 캐시. 주소 변환 가속. hit 0.5~1cy, miss 10~100cy
TPU (Tensor Processing Unit)	Google 개발 AI/ML 전용 ASIC. Systolic Array로 행렬 곱셈 특화. GPU보다 전력 효율↑. Domain-Specific Architecture(DSA)의 대표 사례
Virtual Memory	메인메모리를 디스크의 캐시로 사용. 프로그램별 가상 주소 공간, 보호, Page Table로 변환
Superscalar	Dynamic multiple issue. CPU가 런타임에 사이클당 여러 명령어 발행. OoO 실행 가능
Speculation	분기 결과 등을 예측하여 미리 실행. 맞으면 commit, 틀리면 flush+rollback
TCO (Total Cost of Ownership)	총 소유 비용 = CAPEX + OPEX. 시스템의 전체 수명 동안 발생하는 모든 비용. 성능 평가 시 단순 구매가가 아닌 TCO 기준이 실질적 TCO 단위당 성능. 실질적 가성비 지

perf/TCO	표. 전력 효율이 좋은 시스템은 OPEX↓→TCO↓→perf/TCO↑
Yield	웨이퍼당 정상 다이 비율. 면적↑→수율↓→비용 비선형↑
ALU (Arithmetic Logic Unit)	Arithmetic Logic Unit. CPU 내 산술/논리 연산 수행 핵심 HW 유닛
ASCII (American Standard Code for Information Interchange)	American Standard Code for Information Interchange. 7-bit 128문자 인코딩
AVX (Advanced Vector Extensions)	Advanced Vector Extensions. x86 256/512-bit SIMD 확장 명령어 세트
BHT (Branch History Table)	Branch History Table. 분기 예측용 최근 분기 결과 기록 테이블
BTB (Branch Target Buffer)	Branch Target Buffer. 분기 타겟 주소 캐싱. taken branch fetch 지연 제거
CMOS (Complementary Metal-Oxide-Semiconductor)	Complementary MOS. 현대 IC 제조 기본 기술. $Power = C \times V^2 \times f$
CPU (Central Processing Unit)	Central Processing Unit. Datapath + Control로 구성
CUDA (Compute Unified Device Architecture)	Compute Unified Device Architecture. NVIDIA GPGPU 플랫폼/언어
DAXPY (Double-precision A×X Plus Y)	Double-precision A×X Plus Y. 벡터 연산 벤치마크 ($Y = a \times X + Y$)
DDR (Double Data Rate)	Double Data Rate. 클럭 상승/하강 에지 모두 전송하는 DRAM 기술
DGEMM (Double-precision General Matrix Multiply)	Double-precision General Matrix Multiply. 행렬 곱셈 벤치마크 커널
DRAM (Dynamic Random Access Memory)	Dynamic RAM. 커패시터 전하 저장, refresh 필요. 메인 메모리
ECC (Error Correcting Code)	Error Correcting Code. 메모리 오류 검출/정정 코드 (SEC/DEC 등)
FEC (Forward Error Correction)	수신 측이 재전송 없이 자체 에러 정정. 송신 시 중복 정보(redundancy) 추가. ECC(Hamming SEC/DEC)가 대표적 FEC. “Dependability via Redundancy”의 구현
FP (Floating Point)	Floating Point. 부동소수점. IEEE 754로 실수 표현
FPU (Floating Point Unit)	Floating Point Unit. 부동소수점 연산 전용 HW 유닛
FSM (Finite State Machine)	Finite State Machine. 유한 상태 기계. 캐시 컨트롤러 등 제어 로직 구현
FTL (Flash Translation Layer)	Flash Translation Layer. 플래시 wear leveling + 논리-물리 주소 매핑
HLL (High-Level Language)	High-Level Language. 고수준 프로그래밍 언어 (C, Java, Python)
HT (Hyper-Threading)	Hyper-Threading. Intel의 SMT 구현 (1코어 2스레드 동시 실행)
LAN (Local Area Network)	Local Area Network. 건물 내 근거

	리 네트워크 (Ethernet)
LRU (Least Recently Used)	Least Recently Used. 가장 오래 안 쓴 항목을 교체하는 정책
MIPS	(1) Stanford MIPS ISA 또는 (2) Millions of Instr/Sec (성능 지표, ISA/복잡도 무시→신뢰 불가)
MMX (Multi-Media eXtension)	Multi-Media eXtension. x86 초기 64-bit SIMD 확장
MTBF (Mean Time Between Failures)	Mean Time Between Failures = MTTF + MTTR
MTTF (Mean Time To Failure)	Mean Time To Failure. 평균 고장 시간 (신뢰성 지표)
MTTR (Mean Time To Repair)	Mean Time To Repair. 평균 수리 시간
NFC (Near Field Communication)	Near Field Communication. 근거 리 무선 통신
Network Topology	네트워크의 물리적 연결 구조. Bus/Ring/2D Mesh/Hypercube/Fat Tree/Fully Connected. Bisection BW가 핵심 성능 지표
Routing	메시지의 소스→목적지 경로 결정. Deterministic(고정)/Adaptive(동 적). Deadlock-free 보장 필요 (dimension-order 등)
Flow Control	네트워크 내 버퍼·대역폭 할당 방식. Circuit/Packet switching. Store- and-forward/Cut- through(wormhole). Credit- based로 overflow 방지
OoO (Out-of-Order Execution)	Out-of-Order execution. 의존성 없 는 명령어를 프로그램 순서와 무관하 게 먼저 실행. Reservation Station+ROB+Register Renaming으로 구현
In-Order Commit	OoO 실행 결과를 ROB를 통해 반드 시 프로그램 순서대로 확정(commit). Precise Exception과 Speculation rollback을 가능하게 하는 핵심 원칙
ROB (Reorder Buffer)	OoO 프로세서에서 명령어 결과를 프 로그램 순서대로 보관하는 버퍼. Head부터 순차 commit하여 in- order commit 보장. 예측 실패 시 flush로 rollback
OS (Operating System)	Operating System. I/O·메모리·스케 줄링 관리 시스템 소프트웨어
PLP (Pipeline-Level Parallelism)	작업을 여러 단계로 나누어 각 단계를 동시 처리. 5-stage RISC-V 파이프라 인이 대표적. ILP의 기반
PC (Program Counter)	Program Counter. 현재/다음 실행 명령어 주소 저장 레지스터
PMD (Personal Mobile Device)	Personal Mobile Device. 배터리 구 동 모바일 장치 (스마트폰, 태블릿)
	Page Table Entry. 가상→물리 페이 지 매핑 정보 (valid/dirty/reference

PTE (Page Table Entry)	bit)
QDR (Quad Data Rate)	Quad Data Rate. DDR보다 높은 BW의 DRAM (별도 DDR 입출력)
RLP (Request-Level Parallelism)	독립적 요청들을 병렬 처리. 웹서버 HTTP 요청, DB 쿼리 등. WSC/클라우드에서 가장 중요. 통신 오버헤드 최소→확장성 최고
RISC (Reduced Instruction Set Computer)	Reduced Instruction Set Computer. 단순·규칙·고정길이 ISA 설계 철학
Scatter-Gather	벡터 프로세서에서 비연속 메모리 접근 처리. Gather: 인덱스가 가리키는 흩어진 위치에서 모아 로드. Scatter: 흩어진 위치에 저장. Sparse matrix/indirect addressing에 필수
SCAUSE (Supervisor Exception Cause Register)	Supervisor Exception Cause Register. RISC-V 예외 원인 코드 저장
SDC (Silent Data Corruption)	데이터 손상이 발생했으나 시스템이 감지하지 못하는 상황. ECC 범위를 초과하는 다중 비트 에러 등으로 발생. 잘못된 결과가 조용히 전파되어 치명적
SEC/DEC (Single Error Correction / Double Error Detection)	Single Error Correction / Double Error Detection. Hamming 코드 기반
SEPC (Supervisor Exception Program Counter)	Supervisor Exception PC. RISC-V 예외 발생 명령어의 PC 저장
SM (Streaming Multiprocessor)	Streaming Multiprocessor. GPU 내 여러 SP를 묶은 처리 유닛 (NVIDIA)
SMP (Symmetric Multiprocessor)	Symmetric Multiprocessor. 모든 프로세서가 공유 메모리에 동등 접근
SoC (System on Chip)	System on Chip. CPU+GPU+NPU+MemCtrl 등 단일 칩 집적
SP (Streaming Processor)	Streaming Processor. GPU 내 기본 연산 유닛 (NVIDIA)
SRAM (Static Random Access Memory)	Static RAM. 빠르고 비쌈. 캐시에 사용
SRT (Sweeney-Robertson-Tocher)	Sweeney-Robertson-Tocher. 고속 나눗셈 알고리즘 (여러 뒀 비트/단계)
SSE (Streaming SIMD Extensions)	Streaming SIMD Extensions. x86 128-bit SIMD 확장
Strong Scaling	문제 크기 고정, 프로세서 수 증가. Amdahl 적용→순차 부분이 한계. “같은 문제를 더 빨리”
Weak Scaling	프로세서에 비례하여 문제도 키움. 선형 확장 가능. “더 큰 문제를 같은 시간에”
UMA (Uniform Memory Access)	Uniform Memory Access. 모든 프로세서 메모리 접근 시간 동일
VLIW (Very Long Instruction	Very Long Instruction Word. 컴파

Word)	일러가 여러 연산을 하나의 긴 명령어로 패킹
VMM (Virtual Machine Monitor)	Virtual Machine Monitor. 가상→물리 자원 매핑 (하이퍼바이저)
WAN (Wide Area Network)	Wide Area Network. 광역 네트워크 (인터넷)
Warp	GPU에서 32개 스레드의 그룹. SIMD 방식으로 동시 실행 (NVIDIA)
WSC (Warehouse Scale Computer)	Warehouse Scale Computer. 창고 규모 데이터센터 컴퓨팅 시스템

Chapter 1: Computer Abstractions and Technology

(이전 채팅에서 상세 정리 완료 — 60 slides 전체 반영)

핵심 토픽: 무어의 법칙, 컴퓨터 분류 (PC/Server/Super/Embedded/PMD/Cloud), 8 Great Ideas, SW 계층 (App/System/HW), HW 5대 구성 (Input/Output/Processor/Memory/Cache), ISA/ABI/Implementation, IC 제조와 비용 (Yield, Cost/die), 성능 수식 (CPU Time= $IC \times CPI \times T_c$), CPI mix, Power Wall($P=CV^2f$), Amdahl's Law, SPEC 벤치마크, 멀티프로세서 전환

CPU Time 예시: A(2GHz, 10s) → Cycles=20G. B 목표 6s, $1.2 \times \text{cycles}$ → Rate_B=24G/6=4GHz

CPI mix 예시: Class A(CPI=1), B(2), C(3). Seq1(2,1,2)→10cycles/5IC=CPI2.0. Seq2(4,1,1)→9cycles/6IC=CPI1.5 → IC 적다고 항상 빠른 것 아님

Amdahl 예시: 100s 중 80s 개선. $5 \times$ 전체? → $80/n + 20 = 20$ → 불가능!

Power 예시: 85% C, 85% V, 85% f → $P_{\text{new}}/P_{\text{old}} = 0.85 \times 0.85^2 \times 0.85 = 0.85^4 \approx 0.52$

1-추가. perf/TCO — 실질적 가성비 지표

컴퓨터 시스템의 가치를 평가할 때, 단순히 “성능(perf)”이나 “구매 가격”만 보면 안 된다. 실질적 가성비는 **perf/TCO**(총 소유 비용 대비 성능)로 평가한다.

TCO (Total Cost of Ownership) = CAPEX + OPEX

- **CAPEX (Capital Expenditure, 자본적 지출):** 시스템을 도입할 때 발생하는 초기 투자 비용. 서버 구매비, 장비비, 인프라 구축비, 라이선스 비용 등. 일회성.
- **OPEX (Operating Expenditure, 운영 지출):** 시스템을 운영하는 동안 지속적으로 발생하는 비용. 전력 비용, 냉각 비용, 인건비(관리/유지보수), 네트워크 비용, 소프트웨어 유지보수 등.

왜 perf/TCO가 중요한가?

데이터센터에서는 서버 구매비(CAPEX)보다 **전력+냉각(OPEX)**이 TCO의 상당 부분을 차지한다. 따라서: - 저전력 프로세서는 OPEX↓ → TCO↓ → perf/TCO↑ (가성비 향상) - 비싼 고성능 서버라도 전력 효율이 좋으면 perf/TCO가 더 높을 수 있음 - Google이 10-50% 부하에서 주로 운영 → idle 전력이 OPEX에 큰 영향 → Energy Proportionality 문제와 직결

이것이 Ch1의 Power Wall, SPECpower 벤치마크, Energy Proportionality 논의와 직결된다.

perf/TCO ↑ 전략:

- └ perf ↑: 더 빠른 프로세서, 병렬화, 캐시 최적화
- └ TCO ↓:
 - └ CAPEX ↓: 저렴한 HW, 효율적 설계
 - └ OPEX ↓: 저전력($P=CV^2f$), 효율적 냉각, 자동화

Chapter 2: Instructions — Language of the Computer

(이전 채팅에서 상세 정리 완료 — 95 slides 전체 반영)

핵심 토픽: Instruction Set 개념, RISC-V 소개, 산술 연산(3-오퍼랜드), DP1/2/3, 레지스터($32 \times 64b$), 메모리 오퍼랜드(ld/sd, Little Endian, byte addressing), 즉시값(addi), 부호(2's complement, sign extension), 6가지 명령어 포맷(R/I/S/SB/U/UJ), 논리연산(sll/srl/and/or/xor), 분기(beq/bne/blt/bge/bltu/bgeu), 프로시저(jal/jalr, 스택, leaf/non-leaf, 재귀 fact), Memory Layout, 문자 데이터(ASCII/Unicode, lb/lbu/sb), 32-bit 상수(lui+addi), Branch/Jump addressing(PC-relative), 동기화(lr.d/sc.d), 번역 과정(Compiler→Assembler→Linker→Loader), Dynamic Linking, Sort 종합예시, Array vs Pointer, x86 비교, RISC-V Extensions(I/M/A/F/D/C), Fallacies & Pitfalls, SPEC2006 명령어 빈도

R-type 인코딩 예시: add x9,x20,x21 →

0000000|10101|10100|000|01001|0110011 = 0x015A04B3

if-else 예시: bne x22,x23,Else / add x19,x20,x21 / beq x0,x0,Exit /
Else: sub / Exit:

Factorial 예시: sp-=16, sd x1/x10, base→return 1, recursive→jal fact,
mul x10,x10,x6

Chapter 3: Arithmetic for Computers

(이전 채팅에서 상세 정리 완료 — 52 slides 전체 반영)

핵심 토픽: 정수 덧셈/뺄셈 오버플로우 조건, 곱셈 하드웨어(long multiplication, optimized, faster multiplier), 나눗셈(restoring division, SRT), RISC-V mul/mulh/div/rem 명령어, IEEE 754 부동소수점 (Single/Double, Bias, hidden bit, Denormal, $\pm Inf$, NaN), FP 덧셈 4단계 (정렬→덧셈→정규화→반올림), FP 곱셈 5단계(지수 덧셈→가수 곱셈→정규화→반올림→부호), FP Adder HW(4-stage pipeline), RISC-V FP 레지스터 (f0~f31) 및 명령어(fadd.d/fmul.d/fld/fsd/feq.d 등), Guard/Round/Sticky bits, SIMD/Subword Parallelism, SSE2/AVX, Saturating operation, FP Associativity 불성립

FP 변환 예시: $-0.75 = (-1)^1 \times 1.1_2 \times 2^{-1} \rightarrow S=1, \text{Frac}=100\dots0,$
 $\text{Exp}=-1+127=126=01111110_2$

오버플로우 조건: 양수+양수→결과 음수, 음수+음수→결과 양수이면 overflow.
부호가 다른 피연산자끼리는 overflow 불가

FP 덧셈: Step1 지수정렬 → Step2 가수덧셈 → Step3 정규화 → Step4 반올림
FP 곱셈: Step1 지수덧셈 → Step2 가수곱셈 → Step3 정규화 → Step4 반올림
→ Step5 부호결정

Chapter 4: The Processor

(이전 채팅에서 상세 정리 완료 — 129 slides 전체 반영)

핵심 토픽: Logic Design(조합/순차, clocking methodology), Single-cycle Datapath(IF→ID→EX→MEM→WB, critical path=ld), ALU Control(ALUOp+funct→ALU control), 5-Stage Pipeline(IF/ID/EX/MEM/WB, throughput ↑ latency 동일, Speedup≈stages), Pipeline Registers(IF/ID, ID/EX, EX/MEM, MEM/WB), Hazards 3종(Structural→메모리분리, Data→Forwarding/Stall, Control→Branch Prediction), Forwarding 감지 조건(EX/MEM.Rd=ID/EX.Rs1 등), Load-Use Hazard(stall+bubble, code scheduling으로 회피), Branch Prediction(static/dynamic, 1-bit/2-bit, BTB), Exception/Interrupt(SEPC/SCAUSE, flush, handler), ILP(Deeper pipeline, Multiple issue), Static issue(VLIW, dual-issue), Dynamic issue(Superscalar, OoO, Reservation Station, Reorder Buffer), Speculation(branch/load), Loop Unrolling+Register Renaming, ARM A53(in-order 100mW) vs i7(OoO 130W)

Pipeline performance: Single-cycle $T_c=800\text{ps}$ vs Pipelined $T_c=200\text{ps} \rightarrow 4\times$ speedup

Forwarding 조건: EX hazard: EX/MEM.Rd = ID/EX.Rs1 or Rs2 (Rd≠0, RegWrite=1). MEM hazard: MEM/WB.Rd = ID/EX.Rs1 or Rs2 (EX hazard 아닐 때만)

Load-Use stall: ID/EX.MemRead AND (ID/EX.Rd = IF/ID.Rs1 or Rs2) → bubble

Code scheduling 예시: ld 사이에 독립 명령어 끼워 stall 제거 (13→11 cycles)

Dual-issue scheduling: $\text{IPC}=5/4=1.25$. Loop unrolling(4×) 후 $\text{IPC}=14/8=1.75$

Chapter 5: Large and Fast — Exploiting Memory Hierarchy

(이전 채팅에서 상세 정리 완료 — 109 slides 전체 반영)

핵심 토픽: Locality(Temporal/Spatial), 메모리 계층 (SRAM→DRAM→Flash→Disk, 속도·가격·용량 트레이드오프), Memory Wall(DRAM latency 감소 느낌), Direct Mapped/Set-Associative/Fully Associative 캐시, Tag/Valid bit/Index/Offset 주소 분할, Block size 트레이드오프, Write-Through(write buffer)/Write-Back(dirty bit), Write Allocation, $\text{AMAT}=\text{Hit time}+\text{Miss rate}\times\text{Miss penalty}$, Miss 3C's(Compulsory/Capacity/Conflict), Cache design trade-offs(크기/연관도/블록크기), Multilevel cache(L1 latency reducer/L2 BW amplifier), LRU/Random replacement, Dependability(MTTF/MTTR/MTBF/Availability), Hamming SEC/DEC(ECC DRAM), Virtual Machines(VMM), Virtual Memory(Page/Page fault/Page Table/TLB), Address Translation(가상→물리), Memory Protection(supervisor mode), Cache Coherence(Snooping invalidate/Directory), Memory Consistency, SW optimization(blocking/tiling)

AMAT 예시: 1ns clock, hit=1cy, penalty=20cy, miss=5% → $\text{AMAT} = 1+0.05\times 20 = 2\text{ns}$

Cache perf 예시: I-miss=2%, D-miss=4%, penalty=100cy, base CPI=2, 36% ld/st → stall=2+1.44=3.44 → actual CPI=5.44

Multilevel 예시: L1만: $\text{CPI}=1+0.02\times 400=9$. L2 추가(5ns, 0.5% global miss): $\text{CPI}=1+0.02\times 20+0.005\times 400=3.4 \rightarrow 2.6\times$ 향상

3C's: Compulsory(첫접근)→block ↑/prefetch. Capacity→cache ↑.
Conflict→assoc ↑

Chapter 6: Parallel Processors from Client to Cloud

(이전 채팅에서 상세 정리 완료 — 57 slides 전체 반영)

핵심 토픽: 병렬 프로그래밍의 어려움
(partitioning/coordination/communication), Amdahl's Law 병렬 버전
($\text{Speedup} = 1 / ((1-F) + F/N)$), Strong vs Weak scaling, Flynn 분류
(SISD/SIMD/MIMD/SPMD), Vector processor(RISC-V vector, DAXPY 예
시), SIMD(SSE/AVX) vs Vector, Hardware Multithreading(Fine-
grain/Coarse-grain/SMT), Shared Memory(SMP, UMA/NUMA, lock,
Sum Reduction), GPU(GPGPU, SM/SP/Warp, Tesla 구조, CUDA), GPU
vs CPU 비교(캐시/스레드/BW/coherence), Message
Passing(send/receive, Cluster, Grid), Network
Topologies(Bus/Ring/Mesh/N-cube/Fully-connected, bisection BW),
Roofline Model($\text{AI} = \text{FLOPs}/\text{Byte}$, $\text{Attainable} = \min(\text{BW} \times \text{AI}, \text{Peak})$,
memory/compute bound), 병렬 벤치마크
(Linpack/SPLASH/NAS/PARSEC), OpenMP DGEMM

Amdahl 병렬 예시: 100 프로세서 90× speedup? → $F=0.999$ 필요 (순차
0.1%만)

Scaling 예시: 10 scalar + 100 matrix. 100 proc → $\text{Speedup}=10(10\%)$.
10000 matrix → $\text{Speedup}=91(91\%)$

Roofline: Memory bound(AI낮음)→BW 최적화. Compute bound(AI높
음)→FP 최적화. 캐시가 AI를 높임
